Compute Instance

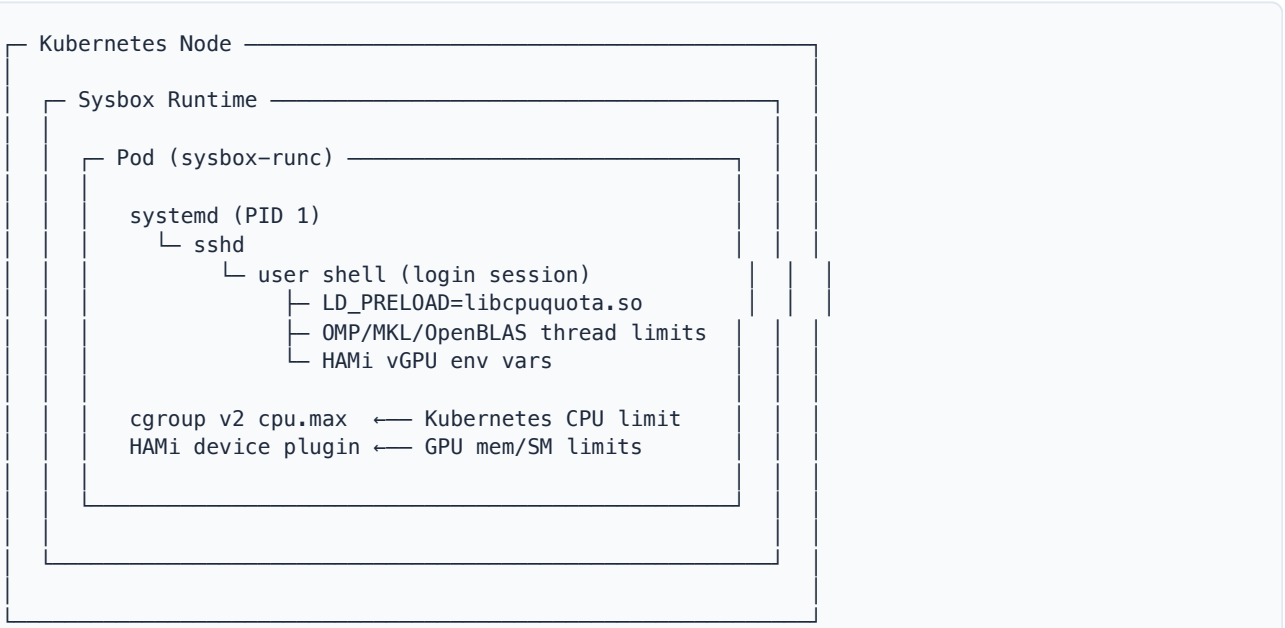
> *This is a Draft / Proposal*

1. Overview

The Compute Instance is a virtual-machine-like resource provided by the platform. It allows users to create isolated virtual machines with configurable CPU, NVIDIA GPU (provided via `nvidia-gpu-operator` / HAMi), memory, storage, and SSH access.

Each project (including default user namespace) is isolated using a dedicated Kubernetes namespace. A namespace can contain multiple tasks, including Compute Instances, Code Server, NeMo, RustCUDA, and user-created storage volumes (PVC).

2. Architecture



Key components:

Component	Role
Sysbox	OCI runtime that provides VM-like isolation. Enables `systemd` as PID 1 inside the container.
`systemd`	Init system inside the pod. Manages `sshd` and other services.
`libcpuquota.so`	`LD_PRELOAD` shim that intercepts CPU-count system calls to respect cgroup v2 quota.
`nproc` wrapper	Shell-level replacement at `/usr/local/bin/nproc` that reads `cpu.max` directly.
HAMi	Kubernetes device plugin for vGPU scheduling (GPU memory and SM core limits).
`entrypoint.sh`	Configures user account, SSH, resource enforcement, and security hardening at boot.

3. Image Evolution

Three iterations of the container image live under `images/`. Each addressed limitations found in the previous version.

3.1 `container-ssh/` — Initial Prototype

A minimal SSH-accessible container running `sshd` in the foreground (`sshd -D`).

- **Base:** `nvidia/cuda:13.0.0-devel-ubuntu24.04`
- **CPU awareness:** `libcpuquota.so` intercepts `sysconf()` only.
- **Thread limits:** Required `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS` to be **set explicitly** as environment variables in the manifest.
- **HAMi:** Forwarded a fixed list of 4 HAMi vars.
- **Init system:** None — ran `sshd -D` directly.

Limitations found: - No `systemd` → no service management, no `systemctl`, no proper init. - `sched_getaffinity()` was not intercepted, so `os.cpu_count()` (Python), `runtime.NumCPU()` (Go), and Java still reported the host CPU count. - Thread-limit env vars had to be manually computed and specified in the pod manifest. - No security hardening — users could bypass resource limits via `sudo`.

3.2 `sysbox-ssh/` — Sysbox + Systemd

Switched to Sysbox runtime with `systemd` as PID 1, providing real VM-like behavior.

- **Base:** `ubuntu:24.04`
- **Init:** `systemd` (`/sbin/init`) with `STOPSIGNAL SIGRTMIN+3`.
- **CPU awareness:** Same `sysconf()`-only shim (still missing `sched_getaffinity()`).

- **Thread limits:** Auto-computed from `nproc` at boot — no longer required in the manifest.
- **HAMi:** Dynamically discovers all `CUDA_DEVICE_MEMORY_LIMIT_*` variables (not just `_0`).
- **PAM sessions:** Added `/etc/environment` propagation for non-login sessions.

Limitations found: - `sched_getaffinity()` still not intercepted. - No security hardening — resource-enforcement files were writable by `sudo`. - No CUDA toolkit in the base image (Ubuntu-only base).

3.3 `sysbox-ssh-kube-cpu-aware/` — Final Image ✅

Adds full CPU-count awareness and security hardening.

- **Base:** `nvidia/cuda:13.0.0-devel-ubuntu24.04` (CUDA toolkit included).
- **CPU awareness:** `libcpuquota.so` now intercepts **both** `sysconf()` and `sched_getaffinity()`, covering Python, Go, Java, OpenMP, and all standard tooling.
- **Thread limits:** Auto-computed; `LD_PRELOAD` enforced via both `/etc/profile.d/` and `/etc/environment`.
- **Security hardening:**
  - Sudoers deny-list prevents writing to cgroup, `libcpuquota.so`, profile scripts, and `/etc/environment`.
  - `chattr +i` makes enforcement files immutable.
  - `readonly` shell variables prevent `unset` of `LD_PRELOAD` and thread limits.
- **Pre-installed tooling:** editors, networking utilities, C/C++ toolchain, Python, `htop`, `gdb`, etc.
- **fail2ban:** Bans IPs after repeated failed SSH login attempts (5 failures in 5 min → 10 min ban).
- **Login banner:** Prints a resource-limits summary table on SSH login.

---

4. CPU Awareness Mechanism

4.1 The Problem

When Kubernetes sets `resources.limits.cpu: "2"`, the kernel enforces the quota via cgroup v2 (`cpu.max` = `200000 100000`), but userspace APIs still report the **host** CPU count. On a 16-core node, `nproc` returns `16`, `os.cpu_count()` returns `16`, and multi-threaded libraries (OpenMP, MKL, OpenBLAS) spawn 16 threads — all competing for 2 cores of CPU time.

4.2 The Solution

Two complementary mechanisms ensure all userspace code sees the correct CPU count:

`libcpuquota.so` (LD_PRELOAD shim)

A shared library loaded via `LD_PRELOAD` that intercepts two system interfaces:

Intercepted call	Used by
`sysconf(_SC_NPROCESSORS_ONLN)`	`nproc(1)`, glibc, OpenMP, MKL, OpenBLAS
`sysconf(_SC_NPROCESSORS_CONF)`	Same as above
`sched_getaffinity()`	Python `os.cpu_count()`, Go `runtime.NumCPU()`, Java

Both overrides read `/sys/fs/cgroup/cpu.max` at call time and compute `ceil(quota / period)`. If the quota is `max` (unlimited) or the file is absent, the real syscall result passes through unchanged.

`nproc` wrapper script

A shell script at `/usr/local/bin/nproc` (shadows `/usr/bin/nproc`) that reads `cpu.max` directly for contexts where `LD_PRELOAD` may not be active:

```
if [ -f /sys/fs/cgroup/cpu.max ]; then
    read -r quota period < /sys/fs/cgroup/cpu.max
    # compute ceil(quota / period) if not "max"
fi
exec /usr/bin/nproc "$@"   # fallback to real nproc
```

Thread-limit environment variables

The entrypoint auto-computes and exports these as `readonly` shell variables:

Variable	Purpose
`OMP_NUM_THREADS`	OpenMP thread count
`MKL_NUM_THREADS`	Intel MKL thread count
`OPENBLAS_NUM_THREADS`	OpenBLAS thread count

These are set in both `/etc/profile.d/compute-env.sh` (login shells) and `/etc/environment` (PAM/non-login sessions).

---

5. GPU Support (HAMi)

GPU resources are managed by HAMi (Heterogeneous AI Computing Virtualization Middleware), which provides vGPU scheduling at the Kubernetes level.

5.1 Resource fields

```
resources:
  limits:
    nvidia.com/gpu: "1"           # Number of vGPUs
    nvidia.com/gpumem: "4096"     # GPU memory limit (MiB)
    nvidia.com/gpucores: "10"     # GPU SM core percentage
```

5.2 Environment variable propagation

HAMi injects environment variables into the container at runtime. The entrypoint dynamically discovers all `CUDA_DEVICE_MEMORY_LIMIT_*` variables and propagates them (along with `CUDA_DEVICE_SM_LIMIT`, `CUDA_DEVICE_MEMORY_SHARED_CACHE`, and `LIBCUDA_LOG_LEVEL`) into:

- `/etc/profile.d/hami.sh` — for login shell sessions.
- `/etc/environment` — for PAM/non-login sessions.

This ensures that GPU limits are visible regardless of how the user connects.

---

6. Security Hardening

The final image prevents users from bypassing resource limits, even with `sudo` access.

6.1 Sudoers deny-list

```
Cmnd_Alias DENY_RESOURCE_BYPASS = /usr/bin/tee /sys/fs/cgroup/*, \
    /usr/bin/tee /etc/profile.d/cpuquota.sh, \
    /usr/bin/tee /etc/profile.d/compute-env.sh, \
    /usr/bin/tee /etc/profile.d/hami.sh, \
    /usr/bin/tee /etc/profile.d/resource-limits.sh, \
    /usr/bin/tee /etc/environment, \
    /usr/bin/tee /etc/fail2ban/jail.local

$USER ALL=(ALL) NOPASSWD: ALL, !DENY_RESOURCE_BYPASS
```

6.2 Immutable file attributes

Critical files are made immutable with `chattr +i`:

- `/usr/local/lib/libcpuquota.so`
- `/usr/local/bin/nproc`
- `/etc/profile.d/cpuquota.sh`
- `/etc/profile.d/compute-env.sh`
- `/etc/profile.d/hami.sh`
- `/etc/profile.d/resource-limits.sh`
- `/etc/environment`
- `/etc/fail2ban/jail.local`

6.3 Readonly shell variables

`LD_PRELOAD`, `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, and `OPENBLAS_NUM_THREADS` are exported as `readonly` in `/etc/profile.d/`, preventing users from unsetting them in their shell session.

6.4 SSH hardening

- Root login is disabled (`PermitRootLogin no`).
- The `root` username is rejected by the entrypoint.
- Password authentication is enabled (credentials set via environment variables).

6.5 fail2ban (SSH brute-force protection)

fail2ban runs as a systemd service and monitors SSH authentication via the systemd journal.

Parameter	Value	Description
`maxretry`	5	Failed login attempts before ban
`findtime`	300 s (5 min)	Window in which failures are counted
`bantime`	600 s (10 min)	Duration of the IP ban
`backend`	`systemd`	Reads auth events from the journal (not log files)

The jail configuration at `/etc/fail2ban/jail.local` is made immutable (`chattr +i`) to prevent users from disabling it.

To check ban status from inside the instance:

```
sudo fail2ban-client status sshd
```

---

7. Pod Manifest

The pod runs under the `sysbox-runc` runtime class with `hostUsers: false` (user-namespace isolation).

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sysbox-hami-test
  labels:
    app: sysbox-hami-test
spec:
  runtimeClassName: sysbox-runc
  hostUsers: false
  containers:
    - name: ubuntu
      image: sysbox-ssh-kube-cpu-aware:24.04
      imagePullPolicy: IfNotPresent
      env:
        - name: INSTANCE_USER
          value: ubuntu
        - name: INSTANCE_PASSWORD
          value: "changeme123"
      ports:
        - name: ssh
          containerPort: 22
      resources:
        limits:
          cpu: "2"
          memory: "2Gi"
          ephemeral-storage: "10Gi"
          nvidia.com/gpu: "1"
          nvidia.com/gpumem: "4096"
          nvidia.com/gpucores: "10"
```

7.1 Configuration reference

Field	Required	Description
`spec.runtimeClassName`	Yes	Must be `sysbox-runc`.
`spec.hostUsers`	Yes	Must be `false` for Sysbox user-namespace isolation.
`INSTANCE_USER`	Yes	Username for the SSH account (cannot be `root`).
`INSTANCE_PASSWORD`	Yes	Password for the SSH account.
`resources.limits.cpu`	Yes	Kubernetes CPU limit. Drives cgroup `cpu.max` and all CPU-count overrides.
`resources.limits.memory`	Yes	Memory limit.
`resources.limits.ephemeral-storage`	No	Ephemeral disk limit.
`nvidia.com/gpu`	No	Number of vGPUs (HAMi).
`nvidia.com/gpumem`	No	GPU memory limit in MiB (HAMi).
`nvidia.com/gpucores`	No	GPU SM core percentage (HAMi).

---

8. Build & Deploy

8.1 Build the image

```
cd images/sysbox-ssh-kube-cpu-aware/
docker build --no-cache -t sysbox-ssh-kube-cpu-aware:24.04 .
```

8.2 Import into containerd (RKE2)

Since the image is built locally and not pushed to a registry, it must be imported into the node's containerd:

```
docker save sysbox-ssh-kube-cpu-aware:24.04 -o /tmp/temp-image.tar

sudo /var/lib/rancher/rke2/bin/ctr \
  --address /run/k3s/containerd/containerd.sock \
  --namespace k8s.io \
  images import /tmp/temp-image.tar
```

Or use the convenience script:

```
bash images/sysbox-ssh-kube-cpu-aware/build-image.sh
```

> **Note:** `imagePullPolicy: IfNotPresent` *in the manifest ensures Kubernetes uses the locally imported image.*

8.3 Deploy the pod

```
kubectl apply -f manifests/pod.yaml
```

8.4 Connect via SSH

```
# Get the pod IP
POD_IP=$(kubectl get pod sysbox-hami-test -o jsonpath='{.status.podIP}')

# SSH in
ssh ubuntu@$POD_IP
# Password: changeme123
```

On login, a resource-limits banner is displayed:

```
┌──────────────────────────────────────────────┐
│  Resource Limits (enforced by Kubernetes)    │
├──────────────────────────────────────────────┤
│  CPU cores : 2                               │
│  OMP threads : 2                             │
│  GPU SM limit : 10                           │
│  GPU mem limit : 4294967296                  │
└──────────────────────────────────────────────┘
```

---

9. File Structure

```
compute-instance/
├── docs/
│   ├── main.md                          # Main architecture and usage guide
│   ├── security-testing.md              # Security testing notes and test plan
│   └── pdf/                             # Exported documentation PDFs
│       ├── generate-pdf.sh              # PDF build script
│       ├── main.pdf
│       ├── security-testing.pdf
│       └── style.css
├── images/
│   ├── container-ssh/                   # v1 — standalone sshd
│   │   ├── Dockerfile
│   │   ├── build-image.sh
│   │   ├── cpuquota.c                   # sysconf()-only shim
│   │   ├── entrypoint.sh
│   │   └── nproc
│   ├── sysbox-ssh/                      # v2 — sysbox + systemd
│   │   ├── Dockerfile
│   │   ├── build-image.sh
│   │   ├── cpuquota.c                   # sysconf()-only shim
│   │   ├── entrypoint.sh
│   │   └── nproc
│   └── sysbox-ssh-kube-cpu-aware/       # v3 — final image ✅
│       ├── Dockerfile
│       ├── README.md
│       ├── build-image.sh
│       ├── cpuquota.c                   # sysconf() + sched_getaffinity()
│       ├── entrypoint.sh
│       ├── jail.local                   # fail2ban SSH jail config
│       └── nproc
├── manifests/
│   └── pod.yaml                         # Example pod manifest
└── testing/
    └── gpu-memory-quota/                # GPU memory quota stress testing
        ├── setup-venv.sh
        ├── stress_test.py
        ├── with-env.sh
        └── without-env.sh
```

10. Prerequisites

Dependency	Version	Purpose
Kubernetes	≥ 1.25	Container orchestration with cgroup v2 support
Sysbox	≥ 0.6	VM-like OCI runtime (enables `systemd`, `runtimeClassName: sysbox-runc`)
HAMi	—	vGPU device plugin for GPU memory/core partitioning
NVIDIA GPU Operator	—	GPU driver and toolkit management
Docker	—	Image building
RKE2 / containerd	—	Container runtime on the nodes

11. Known Limitations

1. **Sudoers deny-list is not exhaustive.** The deny-list blocks `tee` to specific paths, but a sufficiently motivated user with `sudo` could use other tools (e.g., `vi`, `sed`, `dd`) to modify protected files. The `chattr +i` immutable flag is the primary defense; the sudoers deny-list is defense-in-depth.
2. **`LD_PRELOAD` can be bypassed by statically linked binaries.** Programs compiled with `-static` do not load shared libraries and will see the host CPU count.
3. **Single-node local image.** The build script imports the image into the local node's containerd only. For multi-node clusters, the image should be pushed to a container registry.
4. **No SSH key authentication.** The current setup uses password authentication only. SSH key injection (e.g., via a Kubernetes Secret or ConfigMap) is not yet implemented.
5. **(TBD) No persistent storage by default.** The manifest does not mount any PVCs. User data is ephemeral and will be lost when the pod is deleted.